DnD Ultimate Tool Tesztek

László Tamás
Adeniran Ádám Kristóf

Tartalomjegyzék

1. Bevezetés **2**

2. Tesztkörnyezet **2**

3. Tesztesetek **2**

3.1. Login - Unauthorized válasz hiányzó felhasználó esetén 2

3.2. Login - Token visszaadása érvényes felhasználó esetén 3

3.3. Get History - Unauthorized hiányzó identity claim esetén 3

4. Összegzés **4**

1. Bevezetés

A dokumentum célja a rendszer hitelesítési és API működésének tesztelése. A tesztek különböző autentikációs és jogosultsági eseteket fednek le.

2. Tesztkörnyezet

- Backend: C# ASP.Net
- Adatbázis: MySQL
- Teszt eszköz: xUnit Test
- Operációs rendszer: Windows

3. Tesztesetek

3.1. Login - Unauthorized válasz hiányzó felhasználó esetén

Leírás: A rendszernek vissza kell utasítania a bejelentkezést, ha a felhasználó nem létezik.
Elvárt eredmény: HTTP 401 Unauthorized válasz.
Tényleges eredmény: A rendszer helyesen Unauthorized választ adott.

```
[Fact(DisplayName = "Login Returns Unauthorized When User Missing.")]
1 reference
public async Task Login_ReturnsUnauthorized_WhenUserMissing()
{
    var (controller, _) = BuildController(nameof(Login_ReturnsUnauthorized_WhenUserMissing));
    var result = await controller.Login("missing@example.com", "hash");

    var unauthorized = Assert.IsType<UnauthorizedObjectResult>(result);
    Assert.Equal("Invalid credentials.", unauthorized.Value);
}
```

1. ábra. Login válasz nem létező felhasználó esetén

3.2. Login - Token visszaadása érvényes felhasználó esetén

Leírás: A rendszernek token-t kell visszaadnia sikeres autentikáció esetén.
Elvárt eredmény: HTTP 200 OK és egy érvényes token.
Tényleges eredmény: A rendszer sikeresen visszaadta a token-t.

```
public async Task Login_ReturnsToken_ForValidUser()
{
    var (controller, context) = BuildController(nameof(Login_ReturnsToken_ForValidUser));
    var user = new User
    {
        Email = "ok@example.com",
        Username = "ok",
        PasswordHash = Hash("pw"),
        CreatedAt = DateTime.UtcNow,
        IsActive = true,
        Role = "User"
    };
    context.Users.Add(user);
    await context.SaveChangesAsync();

    var result = await controller.Login("ok@example.com", "pw");
    var ok = Assert.IsType<OkObjectResult>(result);

    var tokenProperty = ok.Value?.GetType().GetProperty("token");
    Assert.NotNull(tokenProperty);
    var token = Assert.IsType<string>(tokenProperty!.GetValue(ok.Value)!);
    Assert.False(string.IsNullOrWhiteSpace(token));

    var reloaded = await context.Users.SingleAsync(u => u.Email == "ok@example.com");
    Assert.NotNull(reloaded.LastLoginAt);
}
```

2. ábra. Sikeres login és token visszaadása

3.3. Get History - Unauthorized hiányzó identity claim esetén

Leírás: Az API-nak vissza kell utasítania a kérést, ha hiányzik az identity claim.
Elvárt eredmény: HTTP 401 Unauthorized válasz.
Tényleges eredmény: A rendszer helyesen megtagadta a hozzáférést.

```
[Fact(DisplayName = "Get History returns unauthorized when identity claim is missing.")]
1 reference
public async Task GetHistory_ReturnsUnauthorized_WhenClaimMissing()
{
    var context = TestHelper.CreateContext(nameof(GetHistory_ReturnsUnauthorized_WhenClaimMissing));
    var controller = new DirectMessageController(context)
    {
        ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext()
        }
    };

    var result = await controller.GetHistory(2);

    Assert.IsType<UnauthorizedResult>(result);
}
```

3. ábra. Get history unauthorized válasz identity claim hiányában

4. Összegzés

A tesztek alapján a rendszer megfelelően kezeli:

- Hibás autentikációt
- Érvényes bejelentkezést
- Jogosultság ellenőrzést

A tesztelt funkciók megfelelnek az elvárásoknak.